DMFR-YOLO: Defect Morphology Encoding and Feature Reassembly for Fabric Defect Detection

Kangzhao Tan[#], Jiarui Zhang[#], Junjie Yu, Heng Wu, Jing Lin[*], and Chunhua He[*]

***Abstract*—Circular knitting machines are widely used in continuous fabric production. Needle or yarn abnormalities can cause repeated defects along the fabric before they are detected, resulting in material waste and economic losses. To enable in-process defect detection and intervention, we develop a closed-loop defect detection system with edge inference and wired machine-stop control, and propose a lightweight detector termed DMFR-YOLO. DMFR-YOLO introduces defect morphology encoding (DME) into the backbone to explicitly model horizontally elongated defects, vertically elongated defects, and localized hole-like defects. In the feature pyramid, content-adaptive feature reassembly (CAFR) replaces fixed upsampling with content-adaptive reconstruction, improving the preservation of defect boundaries and weak responses during multi-scale feature fusion. Evaluated on a self-collected fabric defect dataset from a real-world circular knitting machine production line, DMFR-YOLO achieves 66.3% AP and 96.9% AP50, improving the YOLOv8n baseline by 7.0 and 2.8 percentage points, respectively, with 3.39 M parameters. The detector is deployed as an INT8 RKNN model on an RK3588 edge device. The resulting system integrates on-device detection, local stop control, operator review, and cloud-based quality analysis to support intervention during circular-knitting production.**

***Index Terms*—Industrial defect detection, YOLO, deep learning, edge computing, Internet of Things.**

I. Introduction

CIRCULAR knitting machines are important production infrastructure for high-volume weft-knit fabrics used in T-shirts and sportswear [1]–[3]. The International Textile Manufacturers Federation reported approximately 24,500 global shipments of large circular knitting machines in 2025 [4]. Globally, approximately 92 million tonnes of textile waste are generated annually, highlighting the importance of reducing avoidable material losses [5]. Conventional quality checks are often performed after knitting is completed. A persistent needle or yarn abnormality can therefore generate repeated defects along a fabric roll before discovery, accumulating substantial fabric waste and economic losses [6]–[8]. Real-time detection during knitting is needed to identify these defects and enable intervention before further defective fabric is produced.

Recent studies have explored edge computing for fabric defect detection. Song et al. deploy EfficientDet-D0 on an NVIDIA Jetson TX2 for fabric defect detection [9], while Machado et al. deploy YOLO11n on a Jetson Orin Nano for textile defect detection on a fabric rolling line [10]. Yan et al. deploy an improved YOLOv5s model on a Huawei Ascend 310-based Atlas platform for warp-knitted lace defect detection [11]. Beyond textiles, Wang et al. deploy an improved YOLOv5s model on a Jetson Xavier NX for rail surface defect detection [12]. Li et al. deploy LiteYOLO-ID on a Jetson TX2 NX for insulator defect detection [13], and Zhao et al. implement KAM-Net on a Xilinx ZCU102 FPGA for glass defect segmentation [14]. As summarized in Table I, these studies mainly employ general-purpose edge-computing platforms or FPGA development boards. Integrating these platforms into a circular knitting machine requires application-specific camera, sensing, and control interfaces. For installation across many machines, the cost of integrating these interfaces is also a practical consideration, although platform reference prices alone do not establish complete-system cost. This motivates a dedicated edge device that integrates image acquisition, defect detection, and machine-stop control for circular knitting production.

At the algorithm level, YOLO-based detectors provide efficient inference and explicit defect localization [15]. Recent studies have incorporated YOLOv3 into an optical detection system for LCD module defects [16], combined template assistance and attention with YOLOv7 for paper defects [17], and introduced feature prefusion and mask guidance into a Scaled-YOLOv4-style detector for camera-decoration surface defects [18]. Lightweight convolution and structural reparameterization have further supported insulator and rail defect detection on embedded platforms [12], [13]. These works demonstrate the practicality of the YOLO route and improve defect representation or inference efficiency. For circular-knitted fabrics, our focus is to retain this efficiency while explicitly representing the characteristic shapes of defects in repetitive backgrounds.

The defects considered in this work appear as horizontally elongated defects, vertically elongated defects, and localized holes. Standard convolutional feature extraction can learn these patterns implicitly, but it does not explicitly separate their spatial responses. In addition, the nearest-neighbor upsampling used in conventional feature pyramids reconstructs higher-resolution features through fixed replication rather than local content-dependent aggregation. Preserving thin defect boundaries and weak local responses under a limited computational budget therefore motivates morphology-specific feature encoding and adaptive reconstruction during multi-scale fusion.

Based on these hardware and algorithm considerations, we design and implement a dedicated defect detection system for circular knitting machines and propose a lightweight detector, DMFR-YOLO. The system integrates an industrial camera with LED illumination, a Hall sensor, a coaxial video link,

Manuscript received October 12, 2026; revised xxxx xx, 2026; accepted xxxx xx, 2026. ([#]These authors contributed equally: Kangzhao Tan, Jiarui Zhang) ([*]Corresponding authors: Jing Lin and Chunhua He).

Kangzhao Tan, Jiarui Zhang, Junjie Yu, and Chunhua He are with the School of Computer, Guangdong University of Technology, Guangzhou 510006, China (e-mail: tankangzhao@mails.gdut.edu.cn; zhangjiarui1@mail2.gdut.edu.cn; yujunjie2@mails.gdut.edu.cn; hechunhua@pku.edu.cn). Heng Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: heng.wu@foxmail.com). Jing Lin is with the Classroom Management Center, Guangdong University of Technology, Guangzhou 510006, China (e-mail: lj@gdut.edu.cn).

and a self-developed RK3588 edge board. Machine rotation activates detection, and a detected defect triggers a local wired stop command and displays the result for operator review. The cloud receives defect and device records for monitoring and production-quality analysis without participating in the stop decision [19]. Within the detector, defect morphology encoding (DME) introduces horizontal, vertical, and local branches into the backbone, while content-adaptive feature reassembly (CAFR) replaces fixed top-down upsampling with locally predicted neighborhood aggregation. The hardware integration and compact detector jointly support detection and intervention during knitting.

The main contributions of our work are as follows.

(1) We design and implement a dedicated detection system for circular knitting machines, integrating image acquisition, camera transmission, RK3588 edge inference, and local machine-stop control on application-specific hardware, and demonstrate its deployment on a real production line.
(2) We introduce DME to encode the characteristic morphologies of horizontal defects, vertical defects, and holes through horizontal, vertical, and local branches, providing a lightweight morphology-aware representation for circular-knitted defect detection.
(3) We incorporate CAFR into the feature pyramid to replace fixed upsampling with locally predicted reassembly weights, enabling content-adaptive reconstruction during multi-scale fusion.

The remainder of the paper first describes the hardware platform, then develops DMFR-YOLO and its two components. Model conversion and system operation follow the algorithm description. The experimental sections then define the data and evaluation protocol before presenting detector comparisons and component ablations.

II. Hardware Platform

The hardware must provide illuminated fabric imaging, machine-state input, local computation, and a wired stop interface. These requirements establish the operating context for the detector introduced in Section III.

A. Platform Selection and Integration Requirements

Table I summarizes computing platforms used in the defect detection studies reviewed in the introduction. The entries provide hardware context across applications; they do not compare detection accuracy or measured system throughput. The listed platform prices also do not establish the cost of a complete inspection system. Our implementation focuses on integrating the camera link, edge computation, and machine-control interfaces for installation on circular knitting machines.

B. Hardware Configuration

The system combines image acquisition, machine-state sensing, edge computation, and a wired connection to the knitting-machine controller. Fig. 1 shows the camera assembly and our self-developed RK3588 edge-device hardware. The industrial camera and LED light are integrated in the front assembly (A). We use a VEYE MV-MIPI-SC130M monochrome camera with a SmartSens SC130GS-M global-shutter sensor and a 1280×1024 imaging resolution. A camera-side interface board (B) provides a serializer and a coaxial connector, while the RK3588 edge board (C) includes a deserializer for the camera connection. This arrangement connects the camera mounted near the fabric to the edge device responsible for detection and machine control.

The RK3588 edge board integrates flash memory, power management, and input/output interfaces, together with a 4G module, SIM-card slot, and antenna connector for cloud communication. The Hall sensor provides the machine-rotation signal, and a wired connection to the machine controller carries the stop signal. Image processing and the stop decision remain on the edge device; cloud communication is used for device monitoring and defect records rather than for latency-critical control.

C. Frame-Rate Requirement

We determine the required detection frame rate from the circular-knitting geometry and operating speed. For a design diameter of 40 in ($D = 1.016$ m) and a rotation period of $T = 2$ s, the fabric travels approximately $\pi D = 3.192$ m per revolution. The camera is installed approximately 10 cm from the fabric, giving a 16×12 cm field of view with a width of $w = 0.16$ m along the horizontal direction of motion. The minimum frame rate for circumferential coverage is therefore

$$f_{\min} = \frac{\pi D}{T w} = \frac{\pi \times 1.016}{2 \times 0.16} \approx 9.97 \text{ frames/s}. \tag{1}$$

We therefore set the effective detection-throughput requirement to 10 frames/s, corresponding to 20 processed frames per revolution. This requirement guides camera and edge-device selection. Persistent needle or yarn abnormalities can cause defects to propagate as knitting continues, providing further opportunities for detection during subsequent inspection cycles.

D. Signal and Power Connections

Fig. 2 summarizes the interconnections among the system components. Camera data pass through the camera-side serializer, coaxial link, and edge-side deserializer before reaching the RK3588 processor. The Hall sensor supplies the rotation signal through the input/output interface, which also connects the edge device to the knitting-machine controller for local stop control. The power adapter supplies the camera assembly and edge device. The 4G module provides cloud connectivity, while the machine-control path remains local.

TABLE I
PROCESSING PLATFORMS FOR DEFECT DETECTION.

Detection task	Platform	Memory (GB)	Compute capability	Price (USD)
Fabric defects	Jetson TX2 [9]	8	1.33 TFLOPS (FP16)	599
Insulator defects	Jetson TX2 NX [13]	4	1.33 TFLOPS (FP16)	249
Rail surface defects	Jetson Xavier NX [12]	8	21 TOPS (INT8)	399
Textile defects	Jetson Orin Nano [10]	8	40 TOPS (INT8)	499
Warp-knitted lace defects	Huawei Ascend 310 [11]	8	16 TOPS (INT8)	550
Glass defects	Xilinx ZCU102 [14]	4.5†	3.45 TOPS (INT8)‡	3234

†ZCU102 memory is the sum of 4 GB PS DDR4 and 0.5 GB PL DDR4. ‡The 3.45 TOPS value is AMD's reference INT8 peak for three B4096 DPU cores.

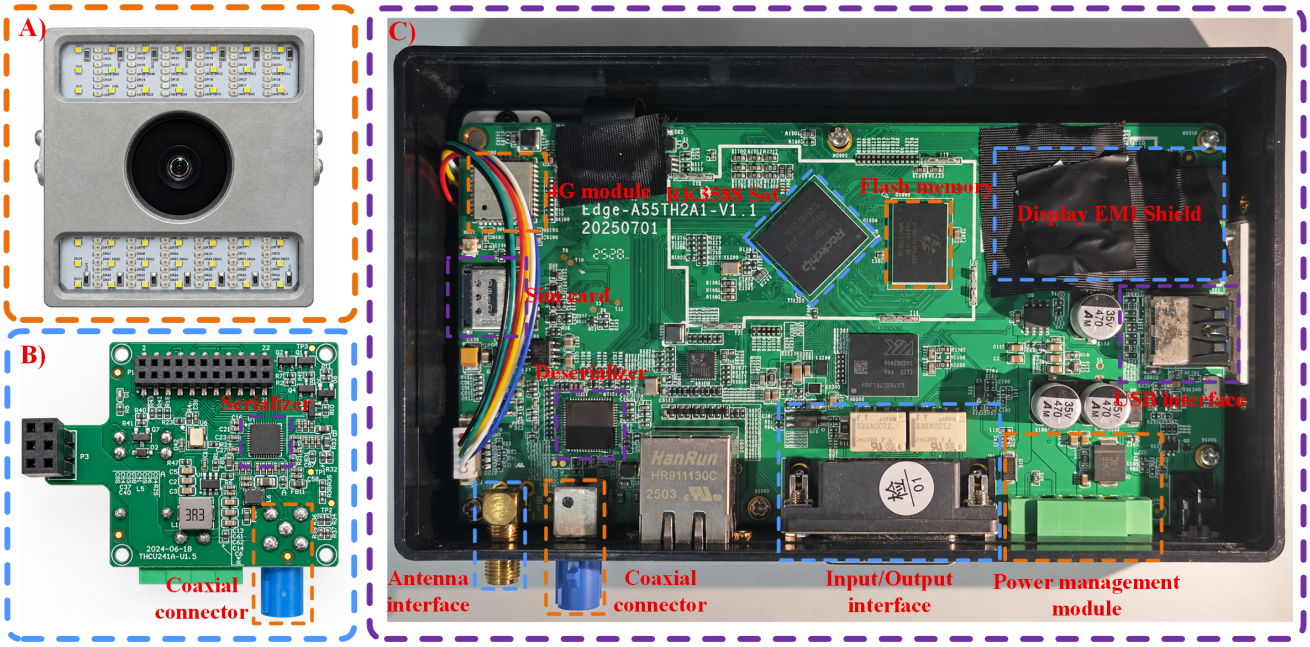


Fig. 1. Hardware of the production-line defect detection system: (A) industrial camera with LED illumination; (B) camera-side interface board with a serializer and coaxial connector; and (C) RK3588 edge board with the labeled communication, memory, camera-interface, input/output, and power-management components.

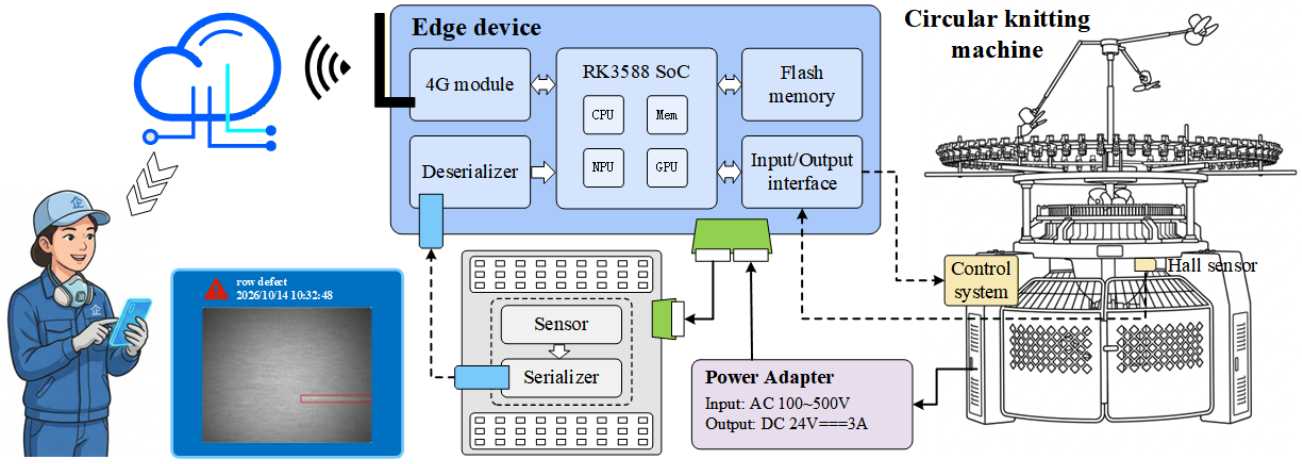


Fig. 2. System component interconnections. The camera stream reaches the RK3588 edge device through the serializer, coaxial link, and deserializer. Wired input/output connections carry the Hall-sensor signal and stop command, while the power adapter supplies the camera and edge hardware.

III. PROPOSED METHOD

A. DMFR-YOLO Architecture

DMFR-YOLO is developed based on YOLOv8n, which provides a compact, modular baseline and an existing RK3588 deployment reference [20], [21]. Within the local-computation setting established by the hardware platform, the proposed modifications focus on defect morphology encoding in the backbone and feature reconstruction in the feature pyramid.

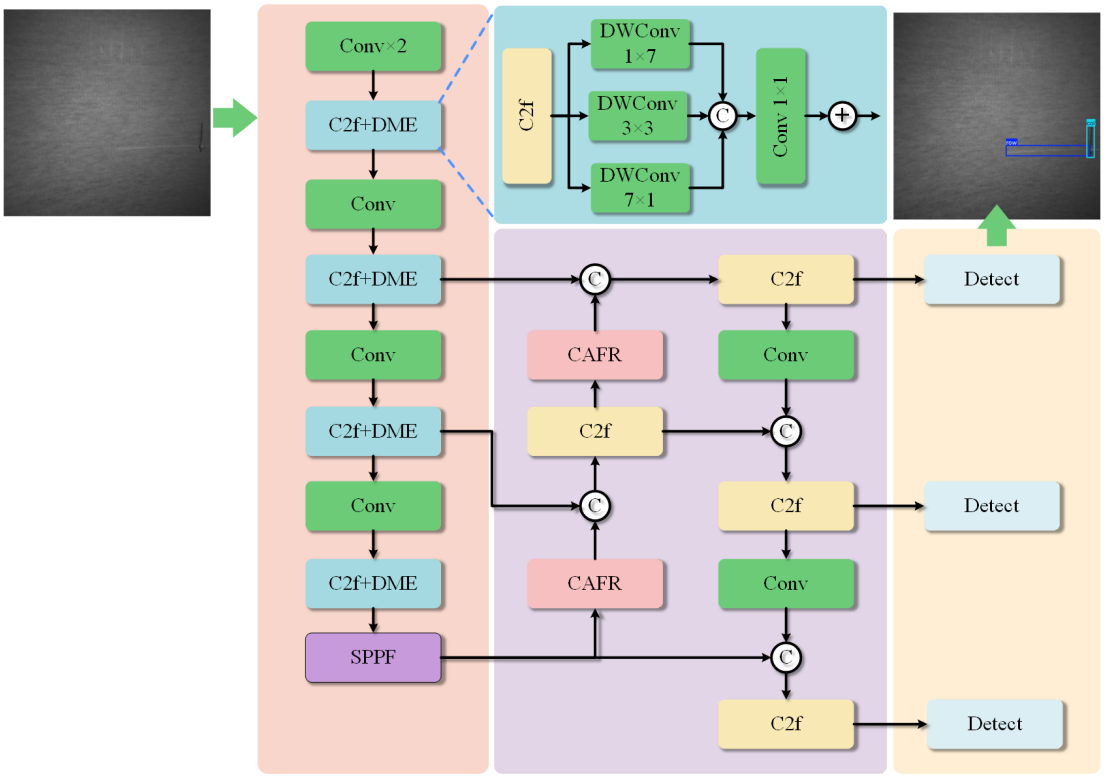


Fig. 3. Architecture of DMFR-YOLO with an enlarged view of DME. DME follows the backbone C2f stages, and CAFR replaces the two top-down upsampling operations. The neck fusion blocks, bottom-up path, and three-scale detection head are retained from YOLOv8n.

Fig. 3 shows the overall architecture. DME is introduced after the main backbone stages to enhance the representation of different defect morphologies. The extracted multi-scale features are then passed to the neck for feature fusion [22], [23]. Within the feature pyramid, CAFR replaces two nearest-neighbor upsampling operations with content-adaptive feature reassembly to better preserve defect boundaries and weak responses. The fused features are subsequently used by the multi-scale detection head to predict defect locations and categories.

B. Defect Morphology Encoding

The three defect categories considered in this work present different spatial characteristics. Horizontal and vertical defects usually extend along one dominant direction, whereas hole defects are more localized. To capture these differences, we introduce a defect morphology encoding (DME) module with three parallel branches, as shown in the enlarged view in Fig. 3. The horizontal and vertical branches use 1×7 and 7×1 depthwise convolutions to enlarge the receptive field along the corresponding spatial directions, while the 3×3 branch captures localized defect structures. Depthwise convolution is adopted to model these spatial patterns with low computational overhead before channel-wise feature fusion.

For a C2f output $X \in \mathbb{R}^{C \times H \times W}$, the three branch responses are

$$F_h = \mathrm{DW}_{1\times7}(X), \tag{2}$$

$$F_v = \mathrm{DW}_{7\times1}(X), \tag{3}$$

$$F_l = \mathrm{DW}_{3\times3}(X). \tag{4}$$

Here, DW denotes depthwise convolution followed by batch normalization and SiLU activation. All branches preserve the spatial dimensions and produce C output channels. Their concatenation is fused as

$$F_d = \phi\left([F_h, F_v, F_l]\right), \tag{5}$$

where $[\cdot]$ denotes channel concatenation and ϕ is a 1×1 convolution mapping $3C$ channels to C, followed by batch normalization and SiLU. This learnable fusion combines the three types of spatial evidence rather than assigning a fixed branch to a particular defect class. A residual connection [24] gives

$$Y = X + F_d. \tag{6}$$

The residual connection combines the morphology-aware features with the incoming C2f output, allowing DME to enhance the backbone representation without discarding the features learned by the preceding layers. The directional branches are based on asymmetric convolution designs [25].

C. Content-Adaptive Feature Reassembly

In the feature pyramid, nearest-neighbor upsampling enlarges feature maps by directly replicating feature values. This fixed operation does not adapt to the spatial structure of defects. To address this limitation, we introduce content-adaptive feature reassembly (CAFR), following the reassembly mechanism of Content-Aware ReAssembly of FEatures (CARAFE) [26]. CAFR predicts spatially varying kernels from the input features and uses them to aggregate local neighborhoods (Fig. 4), aiming

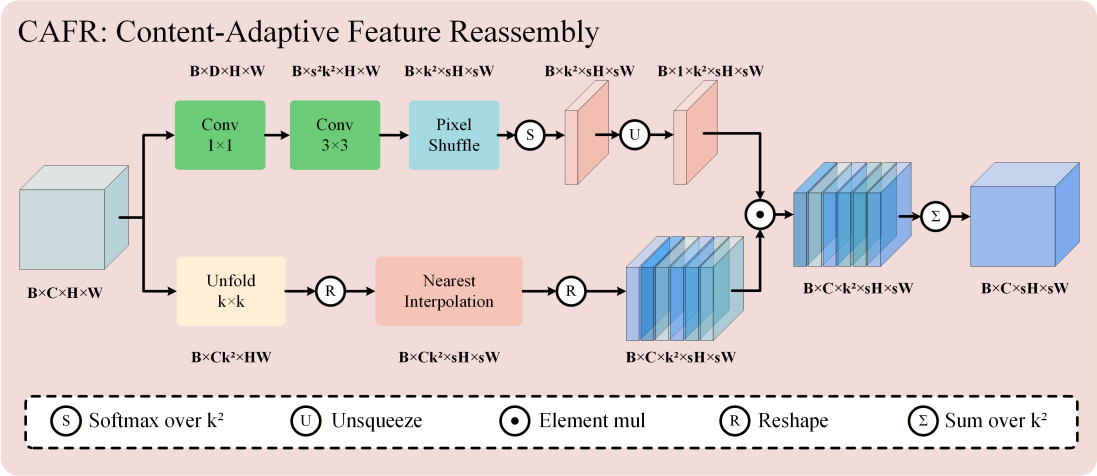


Fig. 4. CAFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds local feature patches and replicates them to align with output locations. Weighted aggregation, rather than this alignment operation, produces the final $2\times$ upsampled feature.

to better preserve defect boundaries and weak responses during multi-scale fusion.

In DMFR-YOLO, CAFR replaces the upsampling operations at both top-down fusion stages, with an upsampling factor of $s = 2$ and a reassembly kernel size of $k = 5$. As shown in Fig. 4, CAFR consists of two parallel branches for kernel prediction and local feature extraction. Their outputs are subsequently combined through weighted aggregation to generate the upsampled features. The two branches and their subsequent feature reassembly are described as follows:

1) Reassembly Kernel Prediction: Given an input feature map $X \in \mathbb{R}^{C\times H\times W}$, a 1×1 convolution first reduces its channel dimension:

$$X_c = E_{1\times1}(X). \tag{7}$$

Here, $X_c \in \mathbb{R}^{D\times H\times W}$, where $D = \max(\lfloor C/4 \rfloor, 16)$ denotes the compressed channel count; $E_{1\times1}$ includes batch normalization and SiLU activation. A linear 3×3 convolution then predicts the unnormalized reassembly weights:

$$K' = E_{3\times3}(X_c). \tag{8}$$

The resulting tensor is $K' \in \mathbb{R}^{s^2k^2\times H\times W}$. For $s = 2$ and $k = 5$, this corresponds to 100 weight channels at each input location. Pixel Shuffle rearranges the predicted weights onto the higher-resolution grid, followed by Softmax normalization:

$$K = \mathrm{Softmax}(\mathrm{PS}(K')). \tag{9}$$

Softmax is applied along the k^2 neighborhood dimension. The resulting tensor $K \in \mathbb{R}^{k^2\times sH\times sW}$ provides a normalized reassembly kernel for each output location.

2) Local Feature Extraction: In parallel, the patch branch extracts local neighborhoods from the input feature map using Unfold. The extracted patches are reshaped and upsampled using nearest-neighbor interpolation to match the spatial resolution of the predicted reassembly kernels:

$$\begin{aligned} P' &= \mathrm{Reshape}(\mathrm{Unfold}_{k\times k}(X)), \\ P &= \mathrm{Reshape}(\mathrm{Nearest}_s(P')). \end{aligned} \tag{10}$$

Here, $\mathrm{Unfold}_{k\times k}$ extracts a $k \times k$ neighborhood around each input position, and $\mathrm{Nearest}_s$ denotes nearest-neighbor upsampling with scale factor $s = 2$. The first reshape produces $P' \in \mathbb{R}^{Ck^2\times H\times W}$; the final reshape gives the aligned patch tensor $P \in \mathbb{R}^{C\times k^2\times sH\times sW}$, where k^2 indexes the positions within each local neighborhood. Nearest-neighbor interpolation is used only for neighborhood alignment, not to generate the final output features.

3) Feature Reassembly: The predicted kernels and the aligned local features are combined through element-wise multiplication and summation along the neighborhood dimension:

$$Y = \sum_{u=1}^{k^2} K_u \odot P_u. \tag{11}$$

Here, $K_u \in \mathbb{R}^{sH\times sW}$ and $P_u \in \mathbb{R}^{C\times sH\times sW}$ denote the weight map and corresponding local feature map for the u-th neighborhood position, respectively. The symbol $\odot$ denotes element-wise multiplication, with the weights broadcast across feature channels, producing an output feature map $Y \in \mathbb{R}^{C\times sH\times sW}$.

By predicting location-specific reassembly weights, CAFR adaptively aggregates local features during upsampling, helping preserve defect-related information in multi-scale feature fusion.

IV. Edge Deployment and System Operation

The detector defined in Section III supplies defect predictions to the hardware platform described in Section II. This section describes the model conversion and its integration into the machine workflow.

A. Model Conversion and On-Device Inference

DMFR-YOLO is deployed locally on the RK3588 edge device. After training, its PyTorch model is exported to ONNX with a fixed 640×640 input, then converted to an RKNN model using RKNN-Toolkit2 [27], with post-training INT8 quantization for efficient NPU execution [28]. An Android 13 application executes the RKNN model on the RK3588 edge device.

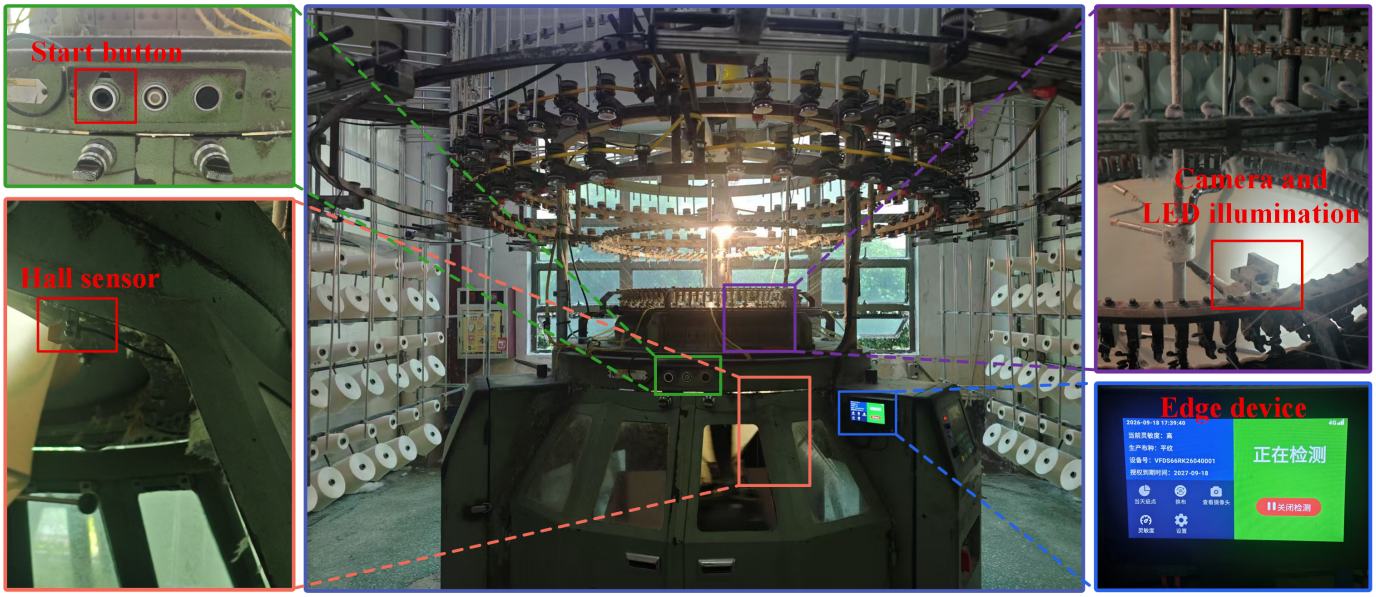


Fig. 5. Actual deployment on the circular knitting machine. The insets highlight the start button, Hall sensor, industrial camera, and RK3588 edge device, respectively.

B. Closed-Loop Detection Workflow

Fig. 5 shows the installation of our defect detection system on a circular knitting machine production line. After the operator starts the circular knitting machine, the Hall sensor detects rotation and signals the RK3588 edge device to switch on the LED light and begin detection from the camera video stream. When a defect is detected, the device sends a stop signal to the machine controller through a wired connection to interrupt further defect propagation. The defect image and detection result are displayed locally for operator review and troubleshooting. The operator resolves the problem and restarts the machine, completing the operational cycle.

C. Cloud Monitoring and Record Management

The cloud platform receives defect records and device states for centralized monitoring, defect statistics, and production-quality analysis. Its interface supports device-state visualization, remote configuration, and review of uploaded defect images. Defect inference and the wired stop decision execute locally, so cloud communication is outside the stop-control path.

V. Experimental Setup

A. Dataset

Images were collected during operation of a real circular-knitting production line. A global-shutter industrial camera captures 1280×1024 frames approximately 10 cm from the fabric, with a field of view of approximately 16×12 cm, where the 16 cm dimension follows the circumferential motion. We selected images with visible defects and manually annotated their bounding boxes. The dataset includes three defect categories: horizontal defects (*row*), vertical defects (*col*), and holes (*hole*).

The dataset contains 403 images with 468 annotated defect instances, divided into training, validation, and test sets of 282, 40, and 81 images, respectively, as summarized in Table II. The validation set is used for checkpoint selection, while performance is reported on the test set containing 91 labeled defects. Because the selected images contain visible defects, this test set evaluates defect localization and classification; it does not measure false-stop frequency during continuous inspection of normal fabric.

TABLE II
IMAGE-LEVEL SPLIT AND DEFECT-INSTANCE COUNTS IN THE REAL PRODUCTION-LINE DATASET.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

B. Implementation Details

Table III summarizes the training and deployment settings. DMFR-YOLO and YOLOv8n are initialized using the pre-trained yolov8n.pt weights, while newly added layers use default initialization. Other comparison models use the available pretrained checkpoints in their respective implementations. The remaining Ultralytics training settings follow the framework defaults.

The comparisons include lightweight YOLO models [20], [29], DETR [30], Deformable DETR [31], Faster R-CNN [32], and an R18 model from the RT-DETRv2 implementation, denoted RT-DETR-R18. All models are evaluated on the same test set using their respective frameworks and evaluation settings. The comparison therefore describes the evaluated model configurations rather than a controlled comparison under identical training and implementation conditions. The AP values in Section VI characterize the test-set detector evaluation; they are not a separate accuracy measurement of the quantized RKNN model.

TABLE III
IMPLEMENTATION PROTOCOL. TRAINING SETTINGS DESCRIBE THE ULTRALYTICS EXPERIMENTS; DEPLOYMENT SETTINGS DESCRIBE THE INSTALLED EDGE DEVICE.

Category	Parameter	Specification
Training setup	Hardware	Intel i7-13700KF; NVIDIA RTX 3090 (24 GB)
	Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Deployment setup	Hardware	Rockchip RK3588
	System	Android 13
	Quantization	INT8
Data protocol	Input resolution	Fixed at 640×640
Training dynamics	Batch	16
	Maximum epochs	300
	Early-stopping patience	80 epochs

C. Evaluation Metrics

We report COCO average precision (AP) across ten IoU thresholds and AP at IoU 0.50 (AP50) [33]. For a predicted box B_p and a ground-truth box B_g, intersection over union is $\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g|/|B_p \cup B_g|$. Let $\mathrm{AP}_c(t)$ denote the average precision for class c at IoU threshold t, and let N_c denote the number of defect categories. The metrics are defined as follows:

$$\mathrm{AP} = \frac{1}{10N_c} \sum_{c=1}^{N_c} \sum_{t \in T} \mathrm{AP}_c(t), \tag{12}$$

$$\mathrm{AP}_{50} = \frac{1}{N_c} \sum_{c=1}^{N_c} \mathrm{AP}_c(0.50), \tag{13}$$

where $T = \{0.50, 0.55, \ldots, 0.95\}$ and $N_c = 3$ in this study. All AP values are reported as percentages. AP therefore evaluates localization over increasingly strict overlap requirements, whereas AP50 uses a single, less restrictive threshold. Differences between reported AP values are expressed in percentage points.

Model parameters and FLOPs are measured using the respective frameworks with an input resolution of 640×640. These quantities describe model size and arithmetic complexity; on-device latency and machine stopping time require separate measurements.

VI. RESULTS AND DISCUSSION

A. Comparison With Other Detectors

Table IV presents the detection results on the test set. DMFR-YOLO achieves 66.3% AP and 96.9% AP50, improving the YOLOv8n baseline by 7.0 and 2.8 percentage points, respectively. These improvements are achieved with only 0.38 M additional parameters and approximately 0.8 G additional FLOPs. The AP gain reflects better aggregate performance across the evaluated IoU thresholds; the aggregate scores alone do not identify the source of the gain.

Among the evaluated lightweight YOLO detectors, DMFR-YOLO achieves the highest AP. Although RT-DETR-R18 attains a slightly higher AP of 67.7%, it requires 20.09 M parameters and 60.72 G FLOPs, compared with 3.39 M parameters and 8.90 G FLOPs for DMFR-YOLO.

TABLE IV
DETECTION RESULTS ON THE TEST SET. BOLD AND UNDERLINED VALUES INDICATE THE HIGHEST AND SECOND-HIGHEST REPORTED SCORES, RESPECTIVELY.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
YOLOv5n	2.50	7.10	93.1	58.4
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	<u>96.8</u>	64.3
Faster R-CNN-R50	41.13	181.45	96.1	64.7
YOLOv8n (Baseline)	3.01	8.10	94.1	59.3
DMFR-YOLO	3.39	8.90	**96.9**	<u>66.3</u>

Fig. 6 compares the detection results of different models on seven selected test images, highlighting differences in detection accuracy and bounding-box localization.

B. Component Ablation

Table V evaluates the individual and combined effects of DME and CAFR on detection performance. Introducing DME increases AP from 59.3% to 64.1%, while CAFR alone improves AP to 60.7%. When both modules are incorporated, DMFR-YOLO achieves 66.3% AP and 96.9% AP50, outperforming either individual modification. Within these evaluated configurations, the combined modification performs better than either module alone. This is consistent with the two modules acting at different stages: DME encodes spatial morphology in the backbone, while CAFR reconstructs features during multi-scale fusion. The aggregate ablation scores do not isolate the contribution of each directional branch or establish statistical significance across repeated training runs.

Fig. 7 compares the feature visualizations of YOLOv8n and DMFR-YOLO. Compared with YOLOv8n, DMFR-YOLO exhibits more concentrated feature responses around elongated defects and holes, with activation regions more closely aligned with the defect shapes. These observations suggest that DMFR-YOLO provides a more focused representation of defect regions.

TABLE V
COMPONENT ABLATION. PARAMETERS ARE IN MILLIONS, FLOPS IN BILLIONS, AND AP50/AP IN PERCENT.

Variant	DME	CAFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ DME	✓	–	3.28	8.8	94.2	64.1
+ CAFR	–	✓	3.11	8.2	94.9	60.7
DMFR-YOLO	✓	✓	3.39	8.9	**96.9**	**66.3**

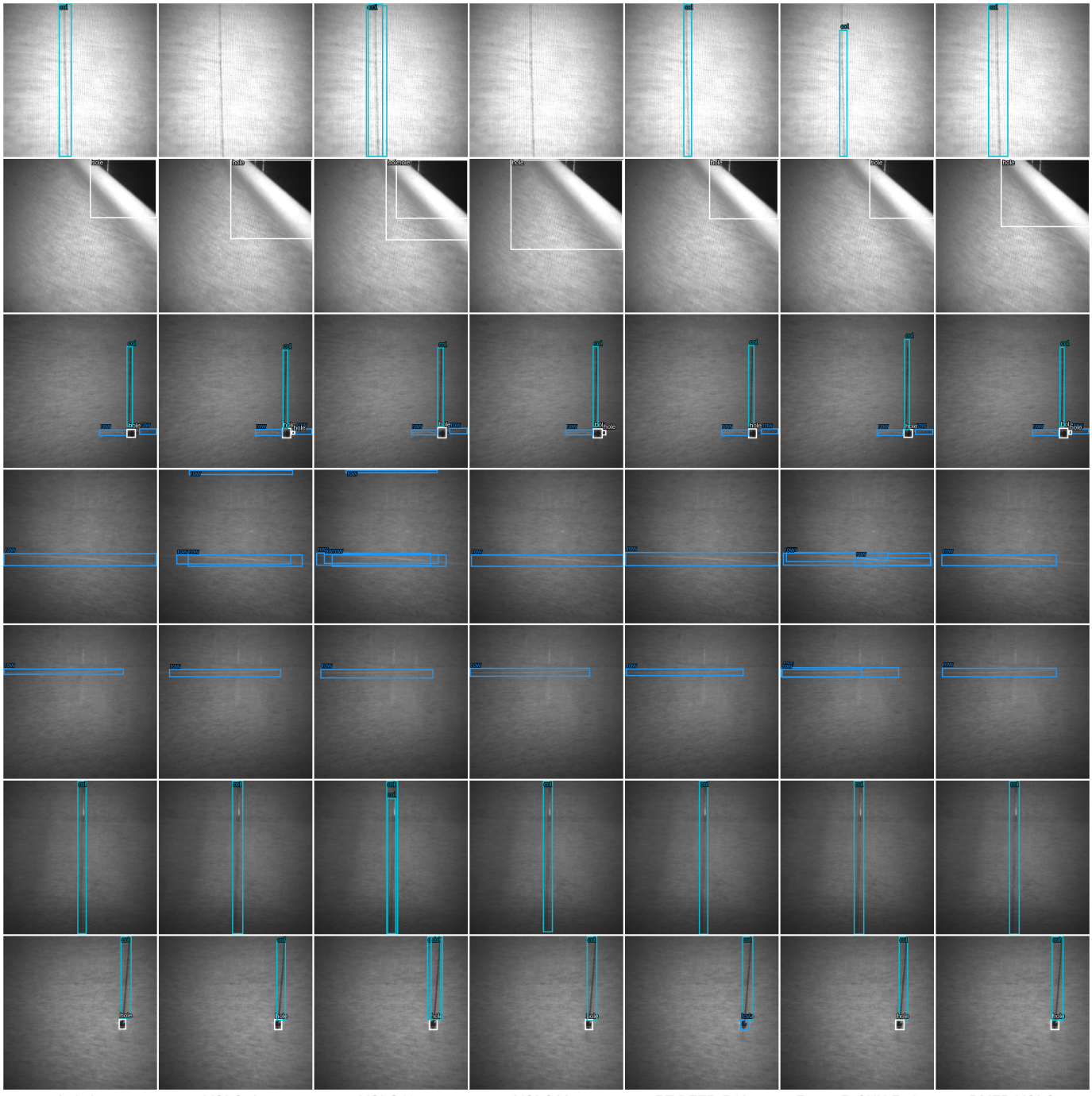


Fig. 6. Qualitative comparison of detection results on selected test images. From left to right, the columns show ground-truth annotations, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and DMFR-YOLO (Ours).

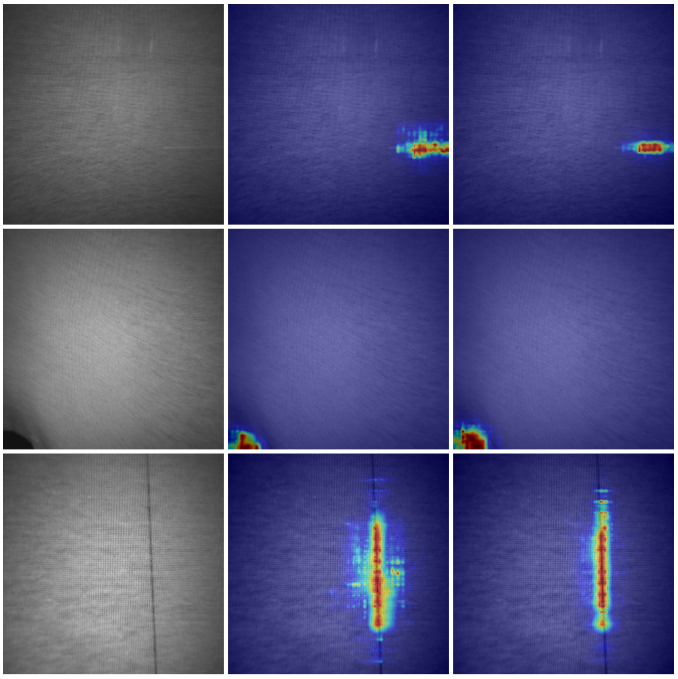


Fig. 7. Feature visualization of YOLOv8n and DMFR-YOLO. From left to right: original image (Origin), YOLOv8n, and DMFR-YOLO.

VII. Conclusion

This paper presents DMFR-YOLO and a defect detection system for circular knitting machines. DMFR-YOLO introduces defect morphology encoding (DME) to capture the spatial characteristics of horizontal defects, vertical defects, and holes. Content-adaptive feature reassembly (CAFR) improves feature reconstruction during multi-scale fusion. Experiments on the production line dataset show that DMFR-YOLO achieves 66.3% AP and 96.9% AP50, outperforming YOLOv8n by 7.0 and 2.8 percentage points, respectively, with 3.39 M parameters and 8.9 G FLOPs. The combined DME and CAFR configuration achieves higher AP than either individual modification in the reported ablation.

The proposed detector is deployed on an RK3588 edge device and integrated with machine monitoring and wired stop control. Defect inference and the stop decision execute locally without relying on cloud communication. Detection results are displayed locally for operators, while the cloud platform supports equipment monitoring and production quality analysis. The reported results establish test-set detection performance and describe the deployment workflow. Quantized-model accuracy, end-to-end response time, physical stopping delay, and false-stop frequency remain to be quantified before drawing conclusions about production performance. Future work will also evaluate generalization across knitted fabrics, texture patterns, and production conditions.

References

[1] Cotton Incorporated, "Knit basics," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-basics/. Accessed: Oct. 8, 2026.

[2] The Woolmark Company, "Circular knit technology," [Online]. Available: https://www.woolmark.com/industry/product-development/processing-innovations/circular-knitting-technology/. Accessed: Oct. 8, 2026.

[3] Cotton Incorporated, "Single and double knits," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/single-and-double-knits/. Accessed: Oct. 8, 2026.

[4] International Textile Manufacturers Federation, "Global textile machinery shipments shrunk in 2025, except for spinning," Jul. 2026, [Online]. Available: https://www.itmf.org/images/dl/press-releases/2026/2026-07-24-ITMSS-48-2025.pdf. Accessed: Oct. 10, 2026.

[5] United Nations Environment Programme, "Transforming the textile industry: Learning to manage waste, better," United Nations Environment Programme, Tech. Rep., Oct. 2025, [Online]. Available: https://wedocs.unep.org/handle/20.500.11822/48707. Accessed: Oct. 14, 2026.

[6] S. Takeuchi, K. Nishioka, H. Uematsu, and S. Tanoue, "Research into development of the defect detection system for knitted fabric produced by the circular knitting machines by image analysis," *Journal of Textile Engineering*, vol. 64, no. 2, pp. 45–49, 2018.

[7] M. M. K. Akter, U. N. Haq, M. M. Islam, and M. A. Uddin, "Textile-apparel manufacturing and material waste management in the circular economy: A conceptual model to achieve sustainable development goal (SDG) 12 for bangladesh," *Cleaner Environmental Systems*, vol. 4, p. 100070, 2022.

[8] Z. Xiang, Y. Shen, M. Ma, and M. Qian, "HookNet: Efficient multiscale context aggregation for high-accuracy detection of fabric defects," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023, art. no. 5016311.

[9] S. Song, J. Jing, Y. Huang, and M. Shi, "EfficientDet for fabric defect detection based on edge computing," *J. Eng. Fibers Fabrics*, vol. 16, pp. 1–13, 2021.

[10] R. Machado, L. A. M. Barros, V. Vieira, F. D. da Silva, H. Costa, and V. Carvalho, "Textile defect detection using artificial intelligence and computer vision—a preliminary deep learning approach," *Electronics*, vol. 14, no. 18, p. 3692, 2025.

[11] T. Yan, Y. Wu, Y. Shi, C. Lin, and L. Ji, "Development and application of an AI visual defect detection system for warp-knitted lace based on 5G+ technology," *Information*, vol. 17, no. 7, p. 623, 2026.

[12] Y. Wang, H. Yu, B. Guo, H. Shi, and Z. Yu, "Research on real-time detection system of rail surface defects based on deep learning," *IEEE Sensors J.*, vol. 24, no. 13, pp. 21 157–21 167, 2024.

[13] D. Li, Y. Lu, Q. Gao, X. Li, X. Yu, and Y. Song, "LiteYOLO-ID: A lightweight object detection network for insulator defect detection," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–12, 2024, art. no. 5023812.

[14] B. Zhao, Y. Wang, H. Zhang, J. Liu, and J. Zhang, "KAM-Net: Kilobyte-scale ultralightweight attention-based network for glass defect detection with algorithm/hardware co-design," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–10, 2025, art. no. 5041310.

[15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.

[16] H. Zhu, J. Huang, H. Liu, Q. Zhou, J. Zhu, and B. Li, "Deep-learning-enabled automatic optical inspection for module-level defects in LCD," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1122–1135, 2022.

[17] X. Li, H. Yan, K. Cui, Z. Li, R. Liu, G. Lu, K. C. Hsieh, X. Liu, and C. Hon, "A novel hybrid YOLO approach for precise paper defect detection with a dual-layer template and an attention mechanism," *IEEE Sensors J.*, vol. 24, no. 7, pp. 11 651–11 669, 2024.

[18] H. Wang, Y. Zhao, F. Zhang, G. Gui, Q. Luo, C. Yang, and W. Gui, "A feature prefusion and mask-guided network for camera decoration defect detection," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–10, 2024, art. no. 5039510.

[19] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[20] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[21] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[22] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[23] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8759–8768.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[25] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.
[26] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.
[27] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.
[28] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.
[29] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," in *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025.
[30] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.
[31] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.
[32] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.
[33] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

Kangzhao Tan received the B.S. degree from Guangzhou University, Guangzhou, China, in 2022. From 2022 to 2024, he worked as a cloud computing R&D engineer with Sangfor Technologies Inc., Shenzhen, China. He is currently pursuing the M.S. degree in computer technology with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include deep learning, computer vision, and industrial defect detection.

Jiarui Zhang is currently a graduate student with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include hardware circuit design and IoT system.

Junjie Yu is currently a graduate student with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include image processing and edge artificial intelligence.

Heng Wu received the B.S. degree in electronic and information engineering from Wuhan University of Science and Technology, Wuhan, China, in 2009, and the M.S. degree in optics and the Ph.D. degree in mechanical manufacturing and automation from South China University of Technology, Guangzhou, China, in 2012 and 2017, respectively.

He is currently an associate professor with the School of Automation, Guangdong University of Technology. His research interests include optical imaging, optical measurement, machine vision, and image processing.

Jing Lin received the B.S. degree in computer science and technology and the M.S. degree in software engineering from Guangdong University of Technology, Guangzhou, China, in 2003 and 2006, respectively.

Since graduation, he has served as an assistant engineer, engineer, and senior engineer. He is currently a senior engineer with the Classroom Management Center, Guangdong University of Technology. His current research interests include information system project management, system informatization, and the intelligent Internet of Things.

Chunhua He received the B.S. degree in microelectronics from Sun Yat-sen University, Guangzhou, China, in 2010, and the M.S. and Ph.D. degrees in microelectronics from Peking University, Beijing, China, in 2013 and 2018, respectively.

From 2013 to 2017, he was an engineer with the No. 5 Electronics Research Institute, Ministry of Industry and Information Technology, Guangzhou, China. From 2017 to 2019, he was an engineer with Midea Group, Foshan, China. From 2019 to 2021, he was a senior engineer with Guangzhou 37 Degree Smart Home Co., Ltd., Guangzhou, China. He joined the School of Computer Science, Guangdong University of Technology, Guangzhou, China, in 2021, where he is currently an associate professor. His current research interests include the design and application of MEMS sensors and artificial intelligence algorithms.